Análisis de un Lenguaje de Programación: Ruby

[redacted]

Universidad del Valle

Modelos y Paradigmas de Programación

Resumen

Los modelos y paradigmas de programación son conceptos fundamentales a la hora de abordar un problema. Conocerlos y entenderlos proporciona herramientas que permiten un juicio objetivo sobre los modelos y técnicas soportadas por un lenguaje, esto abre camino a poder usar de manera correcta los lenguajes en los campos donde mejor se pueden desempeñar. En este artículo se analizará el lenguaje de programación Ruby desde el modelo: declarativo, concurrente, con estado, orientado a objetos y relacional.

Keywords: Modelos de programación, paradigmas de programación, Ruby.

1.Introducción

Ruby es un lenguaje de programación interpretado, reflexivo y orientado a objetos, también puede ser considerado multiparadigma. Este fue desarrollado por Yukihiro "Matz" Matsumoto, el cual presentó este lenguaje en 2006[1]. La sintaxis de este lenguaje fue inspirada a partir de la combinación de varios lenguajes de programación como (Perl, Smalltalk, Eiffel, Ada y Lisp).En Ruby todo es o puede ser un objeto, a diferencia de muchos lenguajes que tienen como datos primitivos los enteros, las cadenas, los booleanos y los arreglos, en Ruby todos estos son vistos como objetos. Todos los objetos heredan de la clase Object y esta a su vez hereda de la clase BasicObject la cual permite crear jerarquías alternativas de objetos[2]. Todo esto permite una gran flexibilidad al desarrollador, ya que tiene la oportunidad de redefinir y crear nuevas funcionalidades con partes ya existentes en el lenguaje.

En la siguientes secciones se analizará el lenguaje de programación Ruby desde los siguientes modelos y paradigmas de programación: **declarativo**, **concurrente**, **con estado**, **concurrente por paso de mensajes**, **orientado a objetos** y **relacional**.

2. Modelo de programación declarativo

2.1 Variables

En Ruby las variables se pueden dividir de acuerdo al alcance léxico, hay cuatro tipo de estas: variables globales, variables de clase, variables de instancia y variables locales. La manera en la que se pueden diferenciar en el código es la siguiente.

- $variable_global
- @@ variable_clase
- @variable_instancia
- variableLocal

Las variables globales son visibles en todo el programa, las variables de clase son visibles para todos los miembros de una clase y el valor de esta variable se comparte, variable de instancia es a lo que llamamos atributo de una clase y esta le pertenece solamente a la instancia de una clase y las variables locales son las que normalmente usamos para

operaciones sencillas dentro de un programa.

Ruby al ser un lenguaje principalmente orientado a objetos, las variables no van a estar restringidas a la ligadura única de un valor, por lo que desde un principio sus variables van a actuar como celdas en Oz[3].

```
variableLocal = 1
p variableLocal
variableLocal = "hola"
p variableLocal
```

```
1
"hola"
```

2.2 Procedimientos y funciones

En Ruby las funciones llamados métodos y los procedimientos no son lo mismo, en primer lugar los métodos no son objetos mientras que los procedimientos sí. Un ejemplo sencillo de estos:

```
proce = proc(){|x,y| x+y}
def metodo(x,y)
  x+y
end
```

Un método y un procedimiento que retornan la suma de dos número, a diferencia de Oz los métodos en Ruby no están obligados a retornar algo, sin embargo si no se especifica con la palabra **return** la salida, entonces la salida es la última expresión evaluada al igual que en las funciones de Oz.
La manera de invocarlos también es diferente:

```
p proce.call(1,3)
p metodo(1,4)
```

```
4
5
```

La programación de alto orden está soportada en Ruby de diferentes maneras, sin embargo los métodos al no ser objetos en Ruby, estos no pueden ser manejados como datos.
Una manera es usando la palabra reservada yield:

```
def print_if(number)
    if yield number
        p number
    end
end
print_if(4){ |el| el%2 == 0 }#imprime 4
print_if(5){ |el| el%2 == 0 }# no imprime nada
```

Aquí yield hace referencia al bloque que está junto a la invocación del método, básicamente el cuerpo de un procedimiento. La siguiente manera es más natural, esta consiste en hacer que un método reciba un procedimiento:

```
def metodoP(p)
  a = p.call(2,4) + 1
  return a
end
p metodoP(proce)
```

```
7
```

Por último veremos cómo se pueden retornar procedimientos con métodos:

```
def metodoRP()
  p "retorna un procedimiento"
  proc(){|x,y| x*y}#multiplica dos variables
end
nuevoP = metodoRP()
p nuevoP.call(4,5)
```

```
"retorna un procedimiento"
20
```

2.3 Funciones recursivas e iterativas

Al igual que en [3], en Ruby también se pueden implementar funciones recursivas e iterativas.
Ejemplo de la función factorial implementada de manera recursiva:

```ruby
#factorial recursivo
def factorialR(n)
  if n == 1
    1
  else
    n*factorialR(n-1)
  end
end
p factorialR(5)
```

Ejemplo de la función factorial implementada de manera iterativa:

```ruby
#factorial iterativo
def factorialI(n,estado)
  if n == 1
    estado
  else
    estado *= n
    factorialI(n-1,estado)
  end
end
p factorialI(5,1)
```

Las dos invocaciones de las funciones dan como resultado:

```
120
120
```

Con todo lo anterior podemos afirmar que Ruby no implementa explícitamente el modelo declarativo, sin embargo el programador puede elegir programar declarativamente al respetar la ligadura única de variables y haciendo uso de las características del modelo funcional anteriormente vistas que provee Ruby.

3. Modelo de programación concurrente

El modelo concurrente se implementa en Ruby con el objeto Thread, el cual permite al igual que en Oz ejecutar instrucciones de manera intercalada o paralela si hay más de un procesador. Sin embargo este modelo está limitado en la manera de comunicar sus hilos, a diferencia de Oz, las variables en Ruby no pueden estar sin ligar porque no existe el bloqueo producto de la espera de la ligadura de una variable, por lo que no se pueden comunicar hilos como hace en Oz.
Un hilo se puede definir de las siguientes maneras:

```ruby
Thread.new {
  p "hola mundo"
  }
Thread.new do
  p "hola mundo 2"
end
thr1 = Thread.new {
  p "hola mundo 3"
}
thr1.join
```

```
"hola mundo 3"
"hola mundo"
"hola mundo 2"
```

Thread es una clase o un objeto?

Como Thread es un objeto entonces este puede ser guardado en una variable, el método **join** se usa para bloquear el hilo principal hasta que el hilo que invoca el método finalice su ejecución, esto se hace ya que en caso de que el hilo principal acabe primero, los hilos diferentes del principal serán finalizados(killed) y no se podrán completar sus instrucciones.

Como las variables no pueden ser usadas como un método de comunicación entre hilos entonces una alternativa desarrollada en Ruby es el objeto Mutex, este garantiza un acceso controlado a las variables que comparten los hilos[4], esto puede usarse para simular el bloqueo por una variable sin ligar en Oz. Se presentan dos ejemplos uno sin usar Mutex y otro usandolo:

Este concepto no aparece en el curso, pues es propio de la concurrencia con estado

```
a = b = 0
t1 = Thread.new {
    a = 1
    sleep 1
    b = 2
    p a + b #3
  }
t2 = Thread.new {
    a = 3
    b = 7
    p a + b#10
}
t1.join
t2.join
```

Posibles resultados:

```
10
3
```

```
10
5
```

Al usar Mutex:

```
a = b = 0
semaphore = Mutex.new
t1 = Thread.new {
  semaphore.synchronize {
    a = 1
    sleep 1
    b = 2
    p a + b #3
  }
  }
t2 = Thread.new {
  semaphore.synchronize {
    a = 3
    b = 7
    p a + b#10
  }
}
t1.join
t2.join
```

Única respuesta:

```
3
10
```

4. Modelo de programación con estado

El modelo con estado está implementado claramente en Ruby, después de todo este es enfocado principalmente a objetos, por lo que el modelo de estado es fundamental. La variables desde un inicio son presentadas con la misma funcionalidad de las celdas.

```
variable = 1
p variable
variable = true
p variable
variable = "hola"
p variable
```

```
1
true
"hola"
```

Teniendo esto en mente podemos dotar a los métodos de un estado explícito al igual que en Oz.

```
@y = 0;
def estado(s)
  if s == "incr"
    @y += 1
  else
    if s == "res"
    @y -= 1
  end
  end
end
estado("incr") # y = 1
estado("incr")# y = 2
p @y
 estado("res") # y = 1
p @y
```

```
2
1
```

Todo lo anterior nos lleva a que Ruby soporta el principio de abstracción porque cuenta con las propiedades de

encapsulación, composicionalidad, instanciación y extensionalidad presentadas en [3].

La encapsulación está dada por el alcance léxico , la programación de alto orden soporta la instanciación y composicionalidad, estas fueron presentadas en la segunda sección, por último el estado explícito que se acaba de presentar soporta la extensionalidad.

5. Modelo de programación concurrente por paso de mensajes

Ruby al no implementar las variables declarativas produce que el procesamiento de mensajes no sea tan natural como lo hace la función **Port** en Oz, lo que Ruby ofrece es el uso de **Socket**[2], esto permite crear una conexión a un host en un puerto dado para después leer la información disponible.

```ruby
require 'socket'
hostname = 'localhost'
port = 2000
 #abre la concexion
s = TCPSocket.open(hostname, port)
# captura las lineas del flujo
while line = s.gets
   puts line.chop # imprime
end
s.close           # cierra la conexion
```

Pero entonces: los sockets son la manera de implementar el concepto de puerto? O sólo de simularlo? O ninguna de las anteriores?

6. Modelo de programación orientado a objetos

El modelo orientado a objetos es el resultado de la abstracción lingüística objeto, esto es posible gracias al modelo con estado. En Ruby las clases se definen de la siguiente manera:

```ruby
class NombreClase < SuperClase
  @@variable_de_clase #puede o no ser declarada
  #inicializador
  def initialize(valor1, valor2... valorN) # N <= M
  #se definen las variables de instancias
  @variable_de_instacia1 = valor1 #valor inicial
  @variable_de_instacia2 = valor2
  ...
  @variable_de_instanciaM = valorM
  end
  #metodos
  def metodo1(valor1..valorN)
    #cuerpo del metodo
  end
  def metodoN(valor1...valorN)
    #cuerpo del metodo
  end
end
```

Un ejemplo en código de la clase Persona:

```ruby
class Persona
  def initialize(nombre, edad, nacionalidad)
  @nombre = nombre
  @edad = edad
  @nacionalidad = nacionalidad
  end
  #metodos
  def decirHola(alguien)
    p "Hola #{alguien}, mi nombre es #{@nombre}"
  end
  def decirEdad
    p "Tengo #{@edad} años"
  end
  def decirNacionalidad
    p "Soy #{@nacionalidad}"
  end
end
```

Instanciación de la clase persona e invocación de sus métodos:

```ruby
p = Persona.new("Jose",21,"Colombiano")
p.decirHola("Juan")
p.decirEdad
p.decirNacionalidad
```

```
"Hola Juan, mi nombre es Jose"
"Tengo 21 años"
"Soy Colombiano"
```

Ruby mantiene los atributos de clase lo más privado posible, esto quiere decir que solamente los propios métodos de la clase pueden acceder a ellos, esto es claramente una práctica de programación orientada a objetos, en Oz pasa lo mismo, los atributos de una clase sólo son accesibles por medio de los métodos de la clase.

Sin embargo Ruby también provee una manera de acceder a los atributos de un objeto sin hacer uso de métodos[2]

```
class Persona
  #attr_reader :nombre
  #attr_writter :nombre
  attr_accessor :nombre
```

- attr_readder se usa para permitir leer el atributo por fuera de la clase.
- attr_writter se usa para modificar el atributo por fuera de la clase.
- attr_accessor permite hacer las dos funciones anteriores.

```
#leer Jose
p p.nombre
 #modificar el nombre con Pedro
p.nombre = "Pedro"
#leer Pedro
p p.nombre
```

```
"Jose"
"Pedro"
```

La herencia que se implemente en Ruby es la herencia simple:

```
class Estudiante < Persona
  def datosEstudiante(codigo, universidad)
    @codigo = codigo
    @univerdidad = universidad
  end
  def decirDatosEstudiante()
    p "Estudio en #{@univerdidad} y mi codigo es #{@codigo}"
  end
end

estu = Estudiante.new("Jose",21,"Colombiano")
estu.datosEstudiante("15478561","Universidad del Valle")
estu.decirHola("Pablo")
estu.decirDatosEstudiante
```

```
"Hola Pablo, mi nombre es Jose"
"Estudio en Universidad del Valle y mi codigo es 15478561"
```

Se crea una nueva clase llama Estudiante que hereda de persona, se define un método y dos variables de instancia nuevas que son **codigo** y **universidad**, después se crea una instancia de la clase Estudiante, en este paso se puede observar que se usa el inicializador de Persona, después se usa el método para guardar los datos del Estudiante y por último se invocan los métodos **decirHola** y **decirDatosEstudiante**, mostrando claramente la herencia.

7. Modelo de programación relacional

Ruby no implementa hasta el momento el concepto de escogencia no-determinista, por lo tanto este modelo no está soportado por Ruby.

8. Conclusiones

A partir del estudio realizado en cada paradigma de programación podemos notar la gran deficiencia que se presenta en los modelos concurrentes, esto es debido a la falta de características que posee el modelo declarativo y su naturaleza orientada a objetos que también dificulta la concurrencia.
Por otro lado vemos el gran poder expresivo que aporta Ruby al hacer uso de la programación funcional y el modelo con estado, el cual se transforma en el orientado objetos y es el paradigma pilar en este lenguaje.

Referencias

[1] Matz. Ruby user's guide URL https://ruby-doc.org/docs/ruby-doc-bundle/UsersGuide/rg/index.html

[2] Ruby documentation . URL https://www.ruby-lang.org/es/documentation

[3] Van-Roy, P., & Haridi, S. (2004). Concepts, techniques, and models of computer programming. MIT press.

[4] Hunt, A , Thomas, D. (2004) Programming Ruby: The Pragmatic Programmer's Guide.Pragmatic Bookshelf; Edición: 2